

DFIR Resilience Framework

Prototype Implementation Plan — 3-Week Build

Pranav · Nasrin · Riya — September 2026

Framing note

Everything below is simulated. No real exploit code, no real malware, no real credential theft — all three modules operate on data structures in Python. “Sysmon-style events,” “PowerShell logs,” etc. mean synthetic records shaped like the real thing, not actual Windows telemetry from a live attack. The core risk this plan guards against is scope creep: the procedural-generation layer can quietly eat the time meant for the degradation/reconstruction/evaluation loop, which is where the actual contribution lives.

1. Prototype scope

One incident family, two evidence sources, one degradation type at two severities, a rule-based reconstruction engine, edge precision/recall/F1 as the primary metric, a 3-tier IR decision rule table, and two plots. Anything beyond this (a second incident family, extra evidence sources, extra degradation types, combined-degradation conditions) is a stretch goal, attempted only after the core loop in week 2 is done and stable.

Must-ship (in priority order)

- Environment model + Family 1 generator producing a valid ground-truth graph
- Evidence generator for process-creation + network/DNS events
- Reconstruction engine that recovers $\blacksquare \approx G$ on full, undegraded evidence (MVP gate)
- Degradation engine: source removal at 2 severities (1 source removed / both removed)
- Reconstruction engine extended to degrade gracefully on missing sources
- Edge precision/recall/F1 metric + a simple confidence score
- 3-tier IR decision rule table (isolate / investigate / monitor) + correctness check vs. ground truth
- `run_experiments.py` over the small matrix + `results.csv`
- 2 plots: F1 vs. % evidence removed; accuracy drop by source removed
- Short written report covering methodology, results, and limitations

Stretch goals (attempt only if ahead of schedule, in this order)

- Random record loss as a second degradation type
- A 2nd incident family
- Auth events as a 3rd evidence source
- Combined-degradation condition (both sources partially degraded at once)
- Confidence-calibration plot (confidence vs. actual accuracy)

Explicitly out of scope

- ML-based reconstruction, adaptive/intelligent attacker planning

- Real VM/Sysmon execution of any kind
- Timestamp-jitter and conflicting-evidence degradation types
- A UI/dashboard beyond static plots
- A full related-work literature review — the report cites the gap in a paragraph, not a section

2. System architecture

Environment Model → Incident Generator → Ground-Truth Graph $G=(V,E)$ [frozen, read-only after this point] → Evidence Generator → Degradation Engine → Reconstruction Engine → Evaluation & IR Decision → results.csv → plots.

The single most important architectural rule: G is generated once, hashed, and never touched again. The reconstruction engine must never hold a Python reference to G — only to files under evidence/. This is a correctness requirement, enforced by an automated test (§7), not a promise.

3. Team responsibilities

Pranav — Environment & Procedural Incident Generation

- **Owns:** environment model (hosts, users, files, services) and the incident generator for Family 1 (weighted-random-walk over a fixed stage sequence, seeded).
- **Builds:** environment.py, actions.py (small action library with pre/effects), generator.py — outputs ground_truth/incident_1.json.
- **Needs:** MITRE ATT&CK stage vocabulary (Initial Access, Execution, Discovery, Credential Access, Lateral Movement, Collection, Exfiltration), networkx, Python's random.Random(seed).
- **Outputs:** ground_truth/incident_1.json + environment/env_1.json, consumed by Nasrin.
- **Depends on:** nothing technically, but the graph JSON schema must be agreed with Nasrin and Riya on day 1.
- **Should not touch:** evidence formatting, degradation, or reconstruction/scoring logic.

Nasrin — Evidence Generation & Degradation

- **Owns:** turning a ground-truth graph into process-creation + network/DNS evidence, and the source-removal degradation engine.
- **Builds:** evidence_gen.py (2 sources only), degrade.py (source removal, 2 severities), a small experiment-config loader.
- **Needs:** Sysmon Event ID 1 shape (process create), enough field names to look realistic; pandas; a per-experiment random seed.
- **Outputs:** evidence/incident_1/full/*.json and evidence/incident_1/degraded/<condition>/*.json plus experiments.csv.
- **Depends on:** Pranav's graph schema being frozen on day 1 — this is the most self-contained module and should move first once the schema exists.

- **Should not touch:** the reconstruction algorithm or scoring — degraded evidence must be produced blind to how the reconstructor works.

Riya — Forensic Reconstruction & Evaluation

- **Owens:** the analysis engine (evidence-only input), scoring, IR-decision mapping, the experiment runner, and the plots.
- **Builds:** reconstruct.py (correlate by host/user/process/time-window into a candidate graph via networkx), score.py (precision/recall + confidence), ir_policy.py (3-tier threshold rules), run_experiments.py, plotting scripts.
- **Needs:** networkx similarity functions, precision/recall/F1, matplotlib.
- **Outputs:** results/results.csv, results/figures/*.png, the methodology/results sections of the report.
- **Depends on:** Nasrin's evidence schema; can build and test the reconstruction engine against Pranav's full, undegraded evidence first — that closes the MVP loop.
- **Should not touch:** the generator's attacker-behavior probabilities or evidence generation, to keep the accuracy numbers non-circular.

4. Technical stack

Layer	Choice	Why
Language	Python 3.11	One language across all three modules removes integration friction.
Graph representation	networkx	Ground truth and reconstruction both become networkx.DiGraph with a shared schema.
Data interchange	JSON on disk between stages	Human-readable, diffable, versioned in git, no DB server.
Tabular results	pandas + CSV	Standard for the results table and figures.
Plotting	matplotlib	Sufficient for line/threshold plots.
Config	A single YAML experiment matrix	Avoids hardcoded loops.
Testing	pytest	Low setup cost; see §7 for the test set.
Version control	Git + GitHub, short-lived feature branches	Folder ownership (§8) avoids merge conflicts.
Reproducibility	--seed threaded through generator → evidence → degradation	Any result can be regenerated exactly.

5. Data architecture

Environment → Incident (seeded generator run) → Ground-Truth Graph $G=(V,E)$, $V = \{\text{user, host, process, file, ip, domain, credential}\}$, $E = \{\text{executed, accessed, created, connected, authenticated, modified}\}$ with a total order on E (the “true timeline”) → Full Evidence (each record carries source, host, user, process, parent, timestamp, target, event_type, and a ref_edge_id linking it back to the G edge that produced it) → Degraded Evidence (minus/plus noise, tagged incident_id/condition/severity/seed) → Reconstruction ■ + confidence $c \in [0,1]$ → Evaluation Result (edge_precision, edge_recall, confidence, ir_decision, ir_decision_correct).

ref_edge_id stays strictly on the generation side of the boundary — it is stripped before Riya’s engine ever sees a file, so there is no way to accidentally cheat off it.

6. Procedural generation, evidence, degradation, reconstruction — build notes

6.1 Incident family

Phishing → PowerShell → Discovery → Collection → HTTPS Exfiltration. A short fixed stage sequence; within a stage the generator picks one concrete action from a small weighted list (e.g. at “Execution,” PowerShell 0.5 / cmd 0.2 / scheduled task 0.3). Preconditions/effects are boolean flags on environment state (has_code_execution, has_credentials, files_collected) — a few dozen lines, not a planner.

6.2 Evidence sources

- Process-creation events (Sysmon-style Event ID 1: process, parent process, command line, host, user, timestamp)
- Network/DNS events (destination IP or domain, host, timestamp)

Each G edge maps to exactly one evidence record via a deterministic rule table (an executed edge always produces one process-creation record; a connected edge produces one network record, plus one DNS record if the target is a domain). This rule table is what makes “realistic” concrete enough to put in the report’s methodology section.

6.3 Degradation

Type	Levels	Mechanism
Source removal	remove process-creation / remove network	drop all records where source == X

Degradation functions are pure: $E' = \text{degrade}(E_{\text{full}}, \text{condition}, \text{severity}, \text{seed})$, never in-place mutation of E_{full} . Every degraded file’s name/metadata encodes exactly which condition/severity/seed produced it, so run_experiments.py can reconstruct the full experiment matrix from the filesystem alone.

6.4 Reconstruction (rule-based, not ML)

- Parse and normalize every evidence record into a common internal shape.
- Correlate records into graph edges by shared keys: same host + user + a time window (records within 5 seconds are causally linked).

- Build **G** as a `networkx.DiGraph`; edge weight = number of independent evidence records supporting that edge (the confidence signal).
- Confidence score = mean normalized edge weight, aggregated to [0,1].
- IR decision mapping: confidence > 0.7 and an exfiltration-stage edge present → “isolate”; confidence 0.4–0.7 → “investigate further”; confidence < 0.4 or no suspicious edges → “monitor / no action.”

Rule-based over ML is a deliberate choice for explainability: when the system fails at a given evidence level, the report can say exactly why.

7. Testing — 3 high-value checks

Keep only the tests that catch the failures most likely to silently corrupt every downstream result:

- **Generator validity (Pranav):** every generated graph is a connected DAG matching Family 1's fixed stage order, and a fixed seed reproduces byte-identical output.
- **Reconstruction regression (Riya):** on full, undegraded evidence, **G** $\approx$ G almost exactly — the strongest single regression test; if this ever fails, a schema changed silently upstream.
- **No-ground-truth-leakage (Riya enforces):** a pytest test that imports `reconstruct.py` and asserts it never imports anything from `ground_truth/` or `environment/`.

Everything else — hand-checked record counts, degradation-severity tolerance checks, scoring-function unit tests on toy graphs — is worth doing only with slack time.

8. Repository structure

`dfir-resilience/environment/` (Pranav) · `generator/` (Pranav) · `ground_truth/` (Pranav's output, read-only after generation) · `evidence_gen/` (Nasrin) · `degrade/` (Nasrin) · `evidence/` (generated, gitignored) · `reconstruct/` (Riya) · `evaluate/` (Riya — the only module allowed to import both `ground_truth` and `reconstruct`) · `experiments/` (Riya, config from Nasrin) · `results/` (generated, gitignored except a checked-in `sample_results/` snapshot) · `tests/` · `report/` · `README.md`. Each person owns a top-level folder; cross-module changes go through a short PR the owner reviews.

9. Three-week schedule (day-level)

Part-time across 3 students, so each week assumes roughly 3–4 working sessions per person, not full-time days.

Week 1 — Schema, generator, evidence, MVP loop closes

Day	Pranav	Nasrin	Riya
Day 1	Agree all 3 JSON schemas with the team (ground-truth graph, evidence record, evaluation result) — nothing else starts before this is frozen	Agree schemas; read Sysmon Event ID 1 / Windows 4624-4688 field shapes	Agree schemas; skim networkx docs for DiGraph + similarity functions
Day 2–3	Build environment.py + actions.py for Family 1	Start evidence_gen.py against a hand-written sample graph (don't block on Pranav)	Sketch reconstruction approach against the frozen schema
Day 3–4	Build generator.py; produce ground_truth/incident_1.json; DAG-validity pytest	Finish process-creation + network generators against Pranav's real output	Start reconstruct.py
Day 5	Support integration; fix schema issues found by Nasrin/Riya	Support integration	Finish reconstruct.py against full evidence
Day 5 — MILESTONE			MVP gate: generate → evidence → reconstruct → one F1 number, run end-to-end
Day 6–7	Buffer; if ahead, sketch a 2nd family	Start degrade.py (source removal only)	Add the no-ground-truth-leakage import test

Week 2 — Degradation, metrics, IR decision, experiment runner

Day	Pranav	Nasrin	Riya
Day 8–9	If ahead of schedule: 2nd incident family. Otherwise: support/buffer	Finish degrade.py (2 severities) + degraded evidence folder tree	Extend reconstruct.py to degrade gracefully on missing sources
Day 10	Buffer / support	Build experiment-config loader (incident × condition × seed matrix)	Add edge precision/recall/F1 + confidence score
Day 11	Buffer / support	Support Riya on evidence schema edge cases	Build ir_policy.py (3-tier rule table) + ir_decision_correct comparison
Day 12–13	Buffer / support	Finalize experiment matrix on disk (~20–30 runs)	Build run_experiments.py; run the small matrix; write results.csv
Day 14 — MILESTONE			Full prototype loop closes: results.csv exists with real numbers across all conditions

Week 3 — Analysis, report, polish

Day	Pranav	Nasrin	Riya
Day 15–16	Write generator/environment methodology paragraph (as it was built, not from memory)	Write evidence/degradation methodology paragraph	Generate the 2 core plots; write results section against results.csv
Day 17	Attempt stretch goals only if the core report is already drafted (2nd family, extra source, combined degradation)	Same — stretch only after core report drafted	Same — stretch metrics only after core report drafted
Day 18–19	Write limitations paragraph (attacker realism, arbitrary weights)	Write limitations paragraph (evidence realism, source coverage)	Write discussion + honest novelty section; assemble report
Day 20	Review/polish repo + README	Review/polish	Finalize one-command reproduction script (make reproduce SEED=...)
Day 21 — MILESTONE			Repo + short report + demo script ready to hand in

10. If it slips — cut in this exact order

- 1st: drop the combined-degradation condition entirely — it was already a stretch goal.
- 2nd: drop the 2nd incident family attempt.
- 3rd: drop the 3rd evidence source (auth) attempt.
- 4th: drop the confidence-calibration plot — keep only the 2 core plots.
- 5th (last resort, and a sign the MVP gate slipped past day 5): reduce the experiment matrix to a single degradation condition per source at 5 seeds, i.e. 2 conditions × 5 seeds = 10 runs, plus baseline.

Do not cut the MVP gate (day 5) or the no-ground-truth-leakage test — both protect against silently invalid results, which is worse than a smaller but valid result set.

11. Deliverables at end of week 3

- A working, tested repository with the structure in §8 and a one-command reproduction path
- results.csv + the 2 core figures (F1 vs. % evidence removed; accuracy drop by source removed)
- A short technical report: problem, methodology, results, limitations, honest novelty assessment — see the companion research plan for its structure
- A short (5 minute) demo script/notebook showing one incident end-to-end